

Week 12 — MongoDB Demo

Installation

- Install MongoDB community edition from <https://www.mongodb.com/docs/manual/installation/>
 - Install pymongo using: `pip install pymongo`
-

1. NoSQL

NoSQL, also referred to as "not only SQL" or "non-SQL", is an approach to database design that enables the storage and querying of data outside the traditional structures found in relational databases. While relational databases organize data into tables with columns and rows, NoSQL databases can store data in a variety of different formats, including document-based, key-value pairs, column-family, and graph-based.

One of the primary advantages of NoSQL databases is their **flexibility**. Because they don't rely on a fixed schema, it's easier to adapt them to changing data structures and requirements. They can also handle larger volumes of data more efficiently, and are often faster at retrieving data than traditional relational databases.

We shall look at the **document-oriented databases**. These store data as documents, typically in JSON (JavaScript Object Notation) or BSON (Binary JavaScript Object Notation) format.

JSON Syntax Rules

JSON is a lightweight format for storing and transporting data. It is "self-describing" and easy to understand.

- JSON data is represented as a collection of **key-value pairs**, where each key is a string and each value can be a string, number, boolean, null, array, or another object.
- Keys and values are separated by a colon (:), and key-value pairs are separated by commas (,).
- Strings are enclosed in double quotes (").
- Arrays are enclosed in square brackets ([]).
- Objects are collections of key-value pairs, enclosed in curly braces ({}), and separated by commas.

Example:

```
{
  "name": "John",
  "age": 30,
  "isStudent": true,   "hobbies": ["reading",
  "running", "hiking"],
  "address": {
    "street": "123 Main St",
```

```
{
  "city": "Anytown",
  "state": "CA",

  "zip": "12345"
}
```

In the context of JSON, a **document** refers to a single JSON object — a collection of key-value pairs enclosed in curly braces. A JSON document is a basic unit of data in MongoDB, analogous to a row in a relational database table.

2. MongoDB

MongoDB is a popular NoSQL document-oriented database that stores data in flexible, JSON-like documents. It is designed to handle large volumes of structured, semi-structured, and unstructured data, and is particularly well-suited for web applications and big data processing.

Key Features of MongoDB:

- **Document Oriented:** Allows for more natural representation of complex data models using JSON-like documents.
 - **No Schema:** Does not require a predefined schema and can thus handle data that is constantly changing.
 - **Querying:** Flexible querying capabilities.
 - **Scalability:** Designed to scale horizontally across multiple servers, making it easier to handle large volumes of data.
-

3. MongoDB Server using Python

We can connect to the MongoDB server using **PyMongo**. PyMongo allows you to connect to a MongoDB database, insert, update, and query data, and perform other operations.

A. Creating a Database

1. Create a MongoClient object:

```
Client = pymongo.MongoClient()
```

2. Use this object to connect to a database that already exists, or create a new one:

```
# Create a new database
database =
Client["newdatabase"]

# Switch to an existing database
database = Client.olddatabase
```

Note: In MongoDB, a database is not created until it gets content. MongoDB waits until you have created a collection with at least one document before it actually creates the database.

3. Display all databases:

```
Client.list_database_names()
```

B. Creating Collections

A **collection** is a group of related documents stored together in a database. Collections are analogous to tables in relational databases, but you can store documents with different structures, fields, and data types in the same collection.

1. Create a collection:

```
collect1 = database["collection1"]
```

Note: A collection is not created until it gets content.

2. Display all collection names:

```
mydbObj.list_collection_names()
```

C. Insert into / Delete from Collections

1. Insert a single record using `insert_one()`. It takes a dictionary and returns an `InsertOneResult` object:

```
newdict = {"key1": "value1", "key2": "value2", "key3":  
"value3"} X = collect1.insert_one(newdict)  
X.inserted_id # returns the id of the inserted document
```

2. Insert multiple records using `insert_many()`. It takes a list of dictionaries and inserts all of them at once. It returns an `InsertManyResult` object:

```
records = [      {"name": "Alice", "age": 22,  
"city": "Delhi"},  
              {"name": "Bob",    "age": 25, "city":  
"Mumbai"},      {"name": "Carol", "age": 21,  
"city": "Chennai"} ]  
result = collect1.insert_many(records)  
result.inserted_ids # returns a list of _id values for all inserted documents
```

Note: This is preferred over calling `insert_one()` in a loop when inserting multiple documents, as it is significantly more efficient.

3. Delete a single record using `delete_one()`. It deletes the first document matching the criteria:

```
result = collect1.delete_one({"key1": "value1"})  
result.deleted_count # returns number of documents deleted
```

4. Delete all matching records using `delete_many()`:

```
result = collect1.delete_many({"key1": "value1"})
```

5. Counting the number of documents:

```
collect1.count_documents({}) # count all documents  
collect1.count_documents({"key2": "value2"}) # count documents matching a condition
```

D. Select / Filter Data from Collections

1. Select all documents:

```
collection1.find() # returns all documents in the collection
```

Notes:

- The `find()` method takes a query object as its parameter specifying conditions that documents must meet.
- This method returns a **cursor object** that can be used to iterate over the matching documents. A cursor is not a list — it gets exhausted after one complete iteration.
- Limit the number of documents returned: `collection1.find(limit=5)`

2. Filter by field value:

```
collection.find({"key2": "value2"})
```

3. Select specific fields (projection):

```
# Method a — using a list
collection.find({"key2": "value2"}, ["key1", "key2"])

# Method b — using a dict with 1s
collection.find({"key2": "value2"}, {"key1": 1, "key2": 1})
```

Note: One must not use 1 and 0 in the same projection object (except if one of the fields is `_id`). Upon specifying one of the fields as 1 (or 0), the rest are assigned 0 (or 1).

4. Querying Nested Documents using Dot Notation

When a document has embedded objects, use dot notation ("`parent.child`") to query fields inside them. Using the example document above with the nested address object:

```
# Find all documents where the city inside address is "Anytown"
collection.find({"address.city": "Anytown"})
```

Note: Writing `{"address": {"city": "Anytown"}}` performs an exact match on the entire address object. It will only return documents where address has solely the city field and nothing else — it will miss documents that also have street, state, zip, etc. Always use dot notation for querying nested fields.

5. Sorting and Skipping with Cursor Methods

After calling `find()`, you can chain cursor methods to sort and paginate results.

- `sort(field, direction)` — sorts results by a field. Use 1 for ascending, -1 for descending.
 - `skip(n)` — skips the first n documents.
 - `limit(n)` — limits results to n documents.
- ```
Sort by age descending, skip first 5, return next 10
collection.find().sort("age", -1).skip(5).limit(10)

Sort by name ascending
collection.find().sort("name", 1)
```

## E. Update Documents in a Collection

1. `update_one(<search_query>, <update_query>)` — updates a single matching document:

```
results = collection.update_one({"key2": "value2"}, {"$set": {"key3": "value11"}})
results.matched_count # number of documents matched
results.modified_count # number of documents actually modified
```

2. `update_many(<search_query>, <update_query>)` — updates all matching documents:

```
results = collection.update_many({"key2": "value2"}, {"$set": {"key3": "value11"}})
```

### 3. `replace_one(<search_query>, <new_document>)` — replaces the entire matched document with a new one:

```
results = collection.replace_one({"key2": "value2"}, {"key3": "value11"})
```

*Note: `replace_one` replaces the entire document. All fields not present in the new document will be permanently removed. It is not the same as `update_one` with `$set`.*

## 4. Additional Update Operators

Beyond `$set`, MongoDB provides several other useful update operators.

`$inc` — increments a numeric field by a given amount:

```
collection.update_one({"name": "Alice"}, {"$inc": {"age": 1}}) # increment age by 1
collection.update_one({"name": "Alice"}, {"$inc": {"score": -5}}) # decrement score by 5
```

`$push` — appends a value to an array field:

```
collection.update_one({"name": "Alice"}, {"$push": {"hobbies": "coding"}})
```

`$pull` — removes a value from an array field:

```
collection.update_one({"name": "Alice"}, {"$pull": {"hobbies": "running"}})
```

## F. Advanced Queries

### 1. Regular Expressions — use the `$regex` operator to match a specific pattern:

```
collect1.find({'key1': {"$regex": "^[v]"}, 'key2': {"$regex": "n$"}}, ['_id'])

key1 starts with 'v' and key 2 ending with n
```

### 2. Comparison Operators — compare values and return documents where the condition is True:

| Operator           | Meaning                  |
|--------------------|--------------------------|
| <code>\$eq</code>  | Equals                   |
| <code>\$ne</code>  | Not equals               |
| <code>\$gt</code>  | Greater than             |
| <code>\$lt</code>  | Less than                |
| <code>\$gte</code> | Greater than or equal to |
| <code>\$lte</code> | Less than or equal to    |

```
collect1.find({'key6': {'$gt': 1}}, ['_id', 'key1', 'key6'])
```

### 3. `$in` and `$all` Operators

`$in` — finds documents where the field value equals any one of the values in the array:

```
collect1.find({'key2': {'$in': ['value5', 'value8']}}, ['_id', 'key2'])
```

`$all` — finds documents where the array field contains all of the specified values:

```
collect1.find({'key7': {'$all': ['A', 'B']}}, ['_id'])
```

### 4. Logical Operators

Logical operators allow combining multiple conditions in a single query.

**\$or** — returns documents that satisfy at least one condition:

```
collection.find({"$or": [{"age": {"$lt": 18}}, {"age": {"$gt": 60}}]})
```

**\$and** — returns documents that satisfy all conditions. Useful when applying multiple conditions on the same field:

```
collection.find({"$and": [{"age": {"$gte": 20}}, {"age": {"$lte": 30}}]})
```

*Note: For different fields, you can put both conditions in the same dictionary: `collection.find({"age": {"$gte": 20}, "city": "Delhi"})`. Use explicit `$and` only when the same field appears more than once in the query.*

**\$nor** — returns documents that fail all the given conditions:

```
collection.find({"$nor": [{"city": "Delhi"}, {"age": {"$gt": 25}}]})
```

## 5. \$elemMatch — Matching Elements in Arrays

When a field is an array, `$elemMatch` matches documents where at least one element in the array satisfies multiple conditions simultaneously.

```
collection.find({"scores": {"$elemMatch": {"$gte": 80, "$lt": 90}}})
```

*Note: Without `$elemMatch`, MongoDB checks conditions across different elements of the array independently. For example, `{"scores": {"$gte": 80, "$lt": 90}}` could match a document with scores `[85, 95]` because `85 >= 80` and `95 < 90` are evaluated on different elements. `$elemMatch` enforces that a single element must satisfy both conditions.*

## G. Aggregation Pipeline

- Executed using the `aggregate()` method on a collection.
- The first argument is a sequence of pipeline stages to be executed.
- An aggregation pipeline operates on all the data in a collection. Each stage is applied to the documents passing through, and whatever documents are emitted from one stage are passed as input to the next stage. `collect1.aggregate(<pipeline>)`

**Example:**

```
pipeline = [{'$match': {'age': {'$gte': 18}}},
{'$group': {'_id': '$gender', 'total': {'$sum': 1}}},
{'$sort': {'total': -1}}, {'$limit': 5}]
result = collection.aggregate(pipeline)
```

The pipeline consists of 4 stages:

- `$match` — filters documents with age field greater than or equal to 18.
- `$group` — groups the remaining documents by their gender field and calculates the total number of documents in each group using `$sum`.
- `$sort` — sorts the groups in descending order based on the value of total.
- `$limit` — limits the output to the first 5 results.
- 

## \$unwind and \$project — Worked Example

`$unwind` deconstructs an array field, creating one output document per array element. This is essential when you want to aggregate over individual items inside arrays.

`$project` reshapes documents — you can include or exclude fields, rename them, or compute new ones.

Example: Given a collection of student documents each having a hobbies array, count how many students have each hobby:

```
pipeline = [# Unwind the hobbies array — one document per
 # hobby per student
 {"$unwind": "$hobbies"},

 # Group by each hobby and count
 {"$group": {"_id":
"$hobbies", "student_count": {"$sum": 1}}},

 # Reshape output — rename _id to hobby_name
 {"$project":
{"hobby_name": "$_id", "student_count": 1, "_id": 0}},

 # Sort by count descending
 {"$sort": {"student_count": -1}}]

result =
collection.aggregate(pipeline) for doc
in result: print(doc)
```

What `$unwind` does step by step: A document `{"name": "John", "hobbies": ["reading", "running"]}` becomes two documents after `$unwind`:

- `{"name": "John", "hobbies": "reading"}`
- `{"name": "John", "hobbies": "running"}`

There are several other aggregation methods one can use including `$skip`, `$avg`, `$max`, and `$min`.

---

## References

- <https://pymongo.readthedocs.io/en/stable/tutorial.html>
- <https://pymongo.readthedocs.io/en/stable/examples/aggregation.html>
- [https://www.w3schools.com/python/python\\_mongodb\\_getstarted.asp](https://www.w3schools.com/python/python_mongodb_getstarted.asp)